

11. 盛最多水的容器

已解答

中等

相关标签

相关企业

提示

给定一个长度为 n 的整数数组 `height`。有 n 条垂线，第 i 条线的两个端点是 $(i, 0)$ 和 $(i, \text{height}[i])$ 。

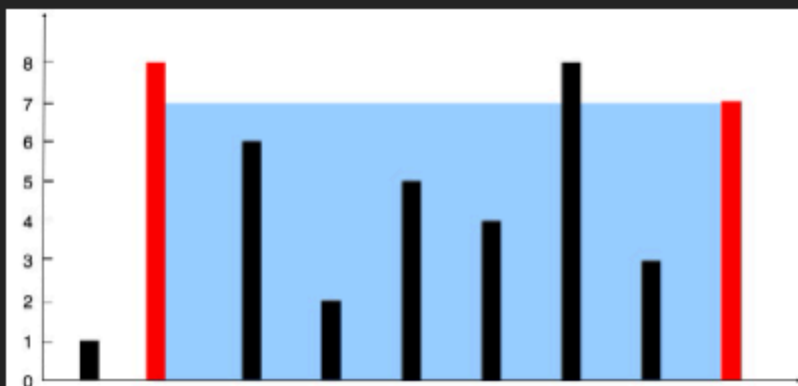
找出其中的两条线，使得它们与 x 轴共同构成的容器可以容纳最多的水。

返回容器可以储存的最大水量。

说明：你不能倾斜容器。

> 来自 <<https://leetcode.cn/problems/container-with-most-water/description/>>

示例 1:



输入: `[1,8,6,2,5,4,8,3,7]`

输出: 49

解释: 图中垂直线代表输入数组 `[1,8,6,2,5,4,8,3,7]`。在此情况下，容器能够容纳水（表示为蓝色部分）的最大值为 49。

示例 2:

输入: `height = [1,1]`

输出: 1

提示:

- `n == height.length`
- `2 <= n <= 10^5`
- `0 <= height[i] <= 10^4`

```
class Solution(object):
    def maxArea(self, height):
        """
        :type height: List[int]
        :rtype: int
        """
        ans = 0
        left = 0
        right = len(height) - 1
        while left < right:
            area = (right - left) * min(height[right], height[left])
            ans = max(area, ans)
            if height[left] < height[right]:
                left += 1
            else:
                right -= 1
        return ans
```

给定 n 个非负整数表示每个宽度为 1 的柱子的高度图，计算按此排列的柱子，下雨之后能接多少雨水。

示例 1:



输入: `height = [0,1,0,2,1,0,1,3,2,1,2,1]`

输出: 6

解释: 上面是由数组 `[0,1,0,2,1,0,1,3,2,1,2,1]` 表示的高度图，在这种情况下，可以接 6 个单位的雨水（蓝色部分表示雨水）。

示例 2:

输入: `height = [4,2,0,3,2,5]`

输出: 9

提示:

- `n == height.length`
- `1 <= n <= 2 * 104`
- `0 <= height[i] <= 105`

```
class Solution(object):
    def trap(self, height):
        """
        :type height: List[int]
        :rtype: int
        """
        n=len(height)
        pre_max= [0] * n
```

//数组声明

```

pre_max[0] = height[0]
for i in range(1,n):
    pre_max[i] = max(pre_max[i-1], height[i])
suf_max = [0] * n
suf_max[-1] = height[-1]
for i in range(n-2, -1, -1):
    suf_max[i] = max(height[i], suf_max[i+1])
ans=0
for h ,pre, suf in zip(height, pre_max, suf_max):
    ans += min(pre, suf) - h

return ans

```

![Exported image](Exported%20image%2020260514133545-2.png)

```

class Solution(object):
    def lengthOfLongestSubstring(self, s):
        """
        :type s: str
        :rtype: int
        """
        ans = 0
        left = 0
        cnt = Counter() #hashmap
        for right, x in enumerate(s):
            cnt[x] += 1
            while cnt[x] > 1:
                cnt[s[left]] -= 1
                left += 1
            ans = max(right - left + 1, ans)
        return ans

```

为什么一定要减 cnt[s[left]]?

因为滑动窗口 [left, right] 缩小时，真正被“踢出去”的是最左边那个字符。
比如字符串：

s = "abba"

我们走到 right = 2 时，窗口是 "abb"。

这时：

cnt['b'] = 2

说明有重复，要缩窗口。

窗口左边 $\text{left} = 0$ ，对应字符是 'a'，所以先移出的是 'a'，不是 'b'。

也就是说应该：

```
cnt['a'] -= 1
```

```
left += 1
```

如果还重复，再继续移。

然后下一次 $\text{left} = 1$ ，这时移出的才是 'b'。

所以缩窗口的逻辑永远是：

```
cnt[s[left]] -= 1
```

```
left += 1
```

不是直接对 x 动手。